Introduction of Arduino and Sensors

Unit 4

Content

- **Introduction of Arduino and Sensors** :
  - Arduino board (pin configuration and description),
  - Basic principle of ultrasonic sensor,
  - Temperature,
  - IR sensor.

What is Arduino?

- A microcontroller board, contains on-board power supply, USB port to communicate with PC, and an Atmel microcontroller chip.
- It simplify the process of creating any control system by providing the standard board that can be programmed and connected to the system without the need to any sophisticated PCB design and implementation.
- It is an open source hardware, any one can get the details of its design & modify it or make his own one himself.

What can it do?

- Sensors (to sense stuff)
  - Push buttons, touch pads, tilt switches.
  - Variable resistors (eg. volume knob / sliders)
  - Photoresistors (sensing light levels)
  - Thermistors (temperature)
  - Ultrasound (proximity range finder)
- Actuators (to do stuff)
  - Lights, LED's
  - Motors
  - Speakers
  - Displays (LCD)

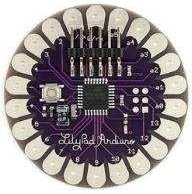
Why Arduino?

- It is Open Source, both in terms of Hardware and Software.
- It is cheap(1300₹), the hardware can be built from components or a prefab board can be purchased for approx. 900₹.
- It can communicate with a computer via serial connection over USB.
- It can be powered from USB or standalone DC power.
- It can run standalone from a computer (chip is programmable) and it has memory (a small amount).
- It can work with both Digital and Analog electronic signals. Sensors and Actuators.
- You can make cool stuff! Some people are even making simple robots.

Different types of Arduino boards:

UNO

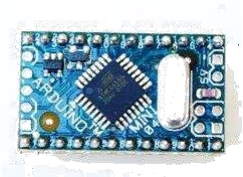
Mega



LilyPad

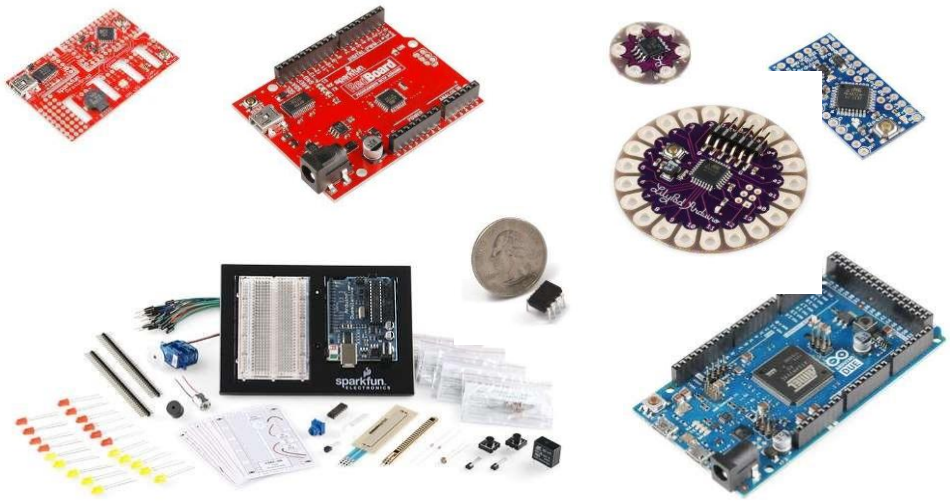
Arduino BT

Arduino Nano

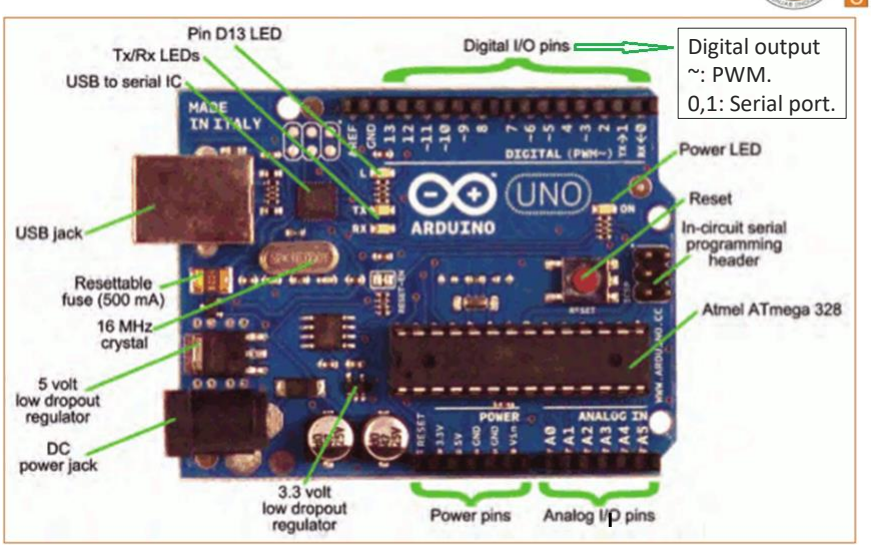


Arduino Mini

Arduino & Arduino compatible boards:

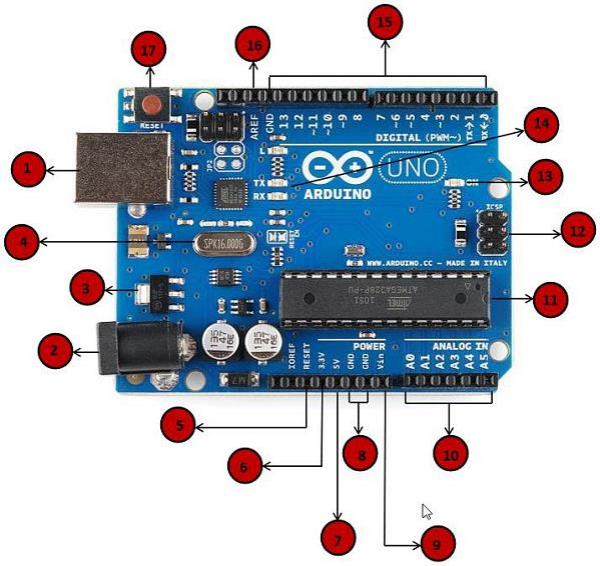


Arduino Uno Board



Connectors/Cables to work with Uno:

Arduino Uno Board Description



1	**Power USB** Arduino board can be powered by using the USB cable from your computer. All you need to do is connect the USB cable to the USB connection (1).
2	**Power (Barrel Jack)** Arduino boards can be powered directly from the AC mains power supply by connecting it to the Barrel Jack (2).
3	**Voltage Regulator** The function of the voltage regulator is to control the voltage given to the Arduino board and stabilize the DC voltages used by the processor and other elements.
4	**Crystal Oscillator** The crystal oscillator helps Arduino in dealing with time issues. How does Arduino calculate time? The answer is, by using the crystal oscillator. The number printed on top of the Arduino crystal is 16.000H9H. It tells us that the frequency is 16,000,000 Hertz or 16 MHz.

5,17	**Arduino Reset** You can reset your Arduino board, i.e., start your program from the beginning. You can reset the UNO board in two ways. First, by using the reset button (17) on the board. Second, you can connect an external reset button to the Arduino pin labelled RESET (5).
6,7 8,9	**Pins (3.3, 5, GND, Vin)** - 3.3V (6) − Supply 3.3 output volt - 5V (7) − Supply 5 output volt - Most of the components used with Arduino board works fine with 3.3 volt and 5 volt. - GND (8)(Ground) − There are several GND pins on the Arduino, any of which can be used to ground your circuit. - Vin (9) − This pin also can be used to power the Arduino board from an external power source, like AC mains power supply.
10	**Analog pins** The Arduino UNO board has six analog input pins A0 through A5. These pins can read the signal from an analog sensor like the humidity sensor or temperature sensor and convert it into a digital value that can be read by the microprocessor.

11	**Main microcontroller** Each Arduino board has its own microcontroller (11). You can assume it as the brain of your board. The main IC (integrated circuit) on the Arduino is slightly different from board to board. The microcontrollers are usually of the ATMEL Company. You must know what IC your board has before loading up a new program from the Arduino IDE. This information is available on the top of the IC. For more details about the IC construction and functions, you can refer to the data sheet.
12	**ICSP pin** Mostly, ICSP (12) is an AVR, a tiny programming header for the Arduino consisting of MOSI, MISO, SCK, RESET, VCC, and GND. It is often referred to as an SPI (Serial Peripheral Interface), which could be considered as an "expansion" of the output. Actually, you are slaving the output device to the master of the SPI bus.
13	**Power LED indicator** This LED should light up when you plug your Arduino into a power source to indicate that your board is powered up correctly. If this light does not turn on, then there is something wrong with the connection.

Arduino Uno Board Description (Cont..)

14	**TX and RX LEDs** On your board, you will find two labels: TX (transmit) and RX (receive). They appear in two places on the Arduino UNO board. First, at the digital pins 0 and 1, to indicate the pins responsible for serial communication. Second, the TX and RX led (13). The TX led flashes with different speed while sending the serial data. The speed of flashing depends on the baud rate used by the board. RX flashes during the receiving process.
15	**Digital I/O** The Arduino UNO board has 14 digital I/O pins (15) (of which 6 provide PWM (Pulse Width Modulation) output. These pins can be configured to work as input digital pins to read logic values (0 or 1) or as digital output pins to drive different modules like LEDs, relays, etc. The pins labeled "~" can be used to generate PWM.
16	**AREF** AREF stands for Analog Reference. It is sometimes, used to set an external reference voltage (between 0 and 5 Volts) as the upper limit for the analog input pins.

INPUT v/s OUTPUT

Referenced from the perspective of the microcontroller (electrical board).

Inputs is a signal / information going into the Arduino board.

Output is any signal exiting the Arduino board.

Examples: Buttons Switches, Light Sensors, Flex Sensors, Humidity Sensors, Temperature Sensors...	Examples: LEDs, DC motor, servo motor, a piezo buzzer, relay, an RGB LED

https://getintopc.com/softwares/3d-cad/proteus-professional-2020-free-download/

Getting Started (Installing Arduino IDE and Setting up Arduino Board)

Check out: http://arduino.cc/en/Reference/HomePage , http://arduino.cc/en/Guide/HomePage .

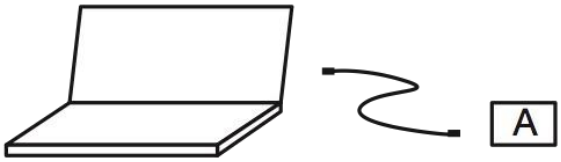
1. Download & install the Arduino environment (IDE) https://www.arduino.cc/en/Main/Software
2. Connect the board to your computer via the USB cable
3. If needed, install the drivers
4. Launch the Arduino IDE
5. Select your board
6. Select your serial port
7. Open the blink example
8. Upload the program

Basic Procedure

- Design the circuit:
  - What are electrical requirements of the sensors or actuators?
  - Identify inputs (analog inputs)
  - Identify digital outputs
- Write the code:
  - Build incrementally
    - Get the simplest piece to work first
    - Add complexity and test at each stage
    - Save and Backup frequently
  - Use variables, not constants
  - Comment liberally

Writing and Uploading Code into Arduino

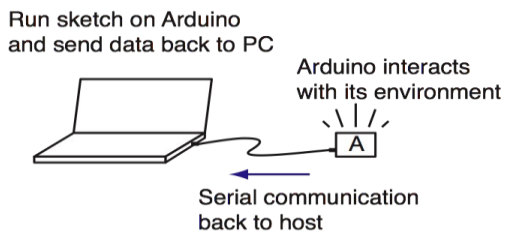
Write sketch on PC



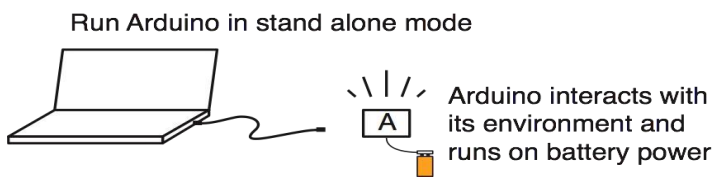
Download sketch to Arduino

Running Code

- Running Code While Tethered



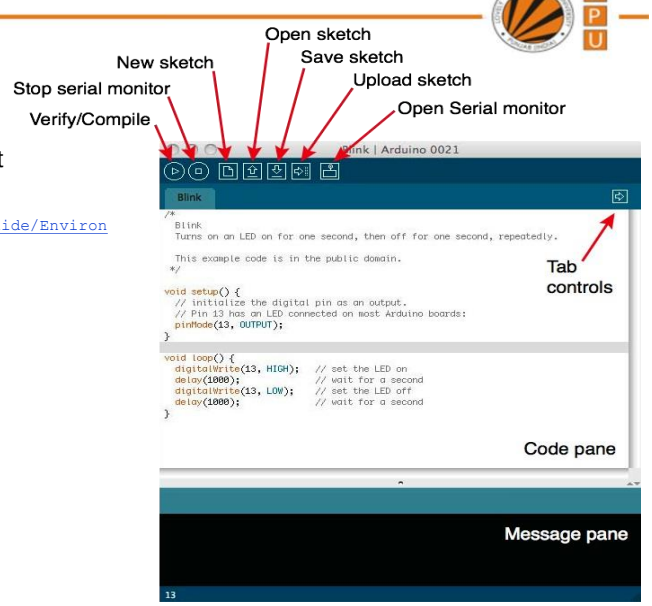
- Running Code Stand-Alone



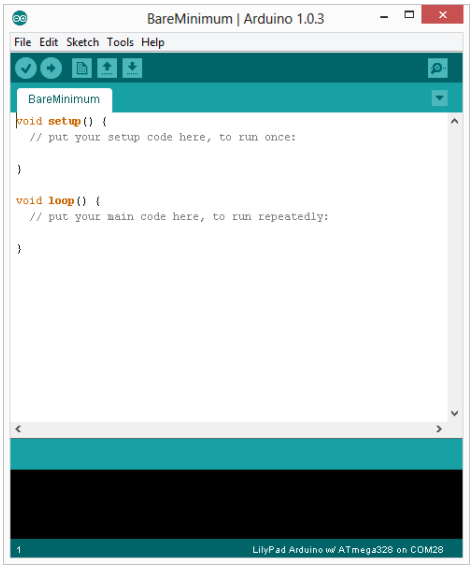
Arduino IDE

IDE =
Integrated Development Environment

http://www.arduino.cc/en/Guide/Environment



Arduino IDE (Cont..)

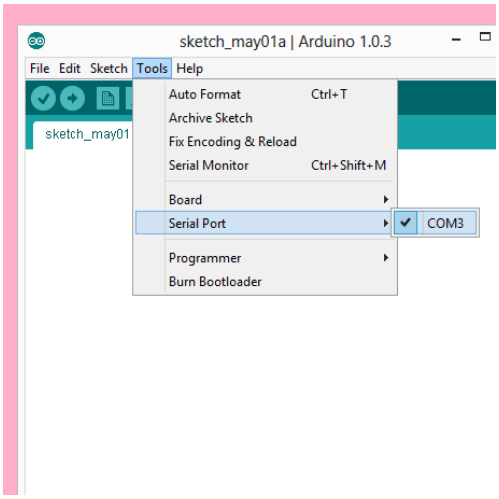


Two required functions / methods / routines:

```
void setup()
{
        // runs once
}

void loop()
{
        // repeats
}
```

Arduino IDE (Cont..)

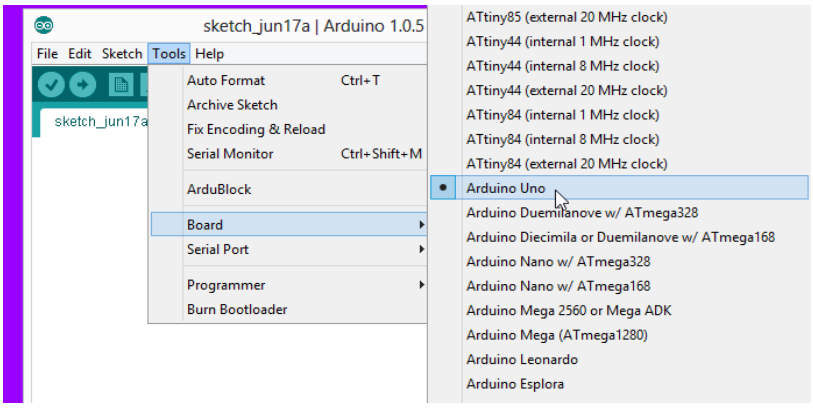


Settings: Tools → Serial Port

Your computer communicates to the Arduino microcontroller via a serial port → through a USB-Serial adapter.

Check to make sure that the drivers are properly installed.

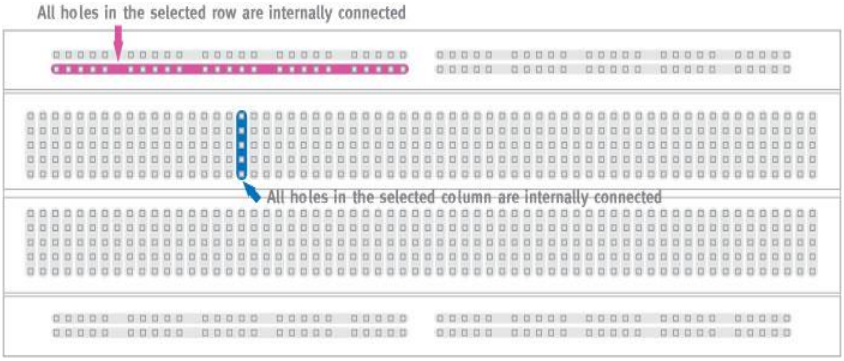
Arduino IDE (Cont..)

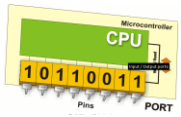


Settings: Tools → Board

Next, double-check that the proper board is selected under the Tools→Board menu.

Breadboard Layout:





Arduino Digital I/0

```
pinMode(pin, mode);
```

Sets pin to either `INPUT` or `OUTPUT`

```
Eg1. pinMode(13, OUTPUT);
```

```
digitalRead(pin);
```

Reads `HIGH` or `LOW` from a pin

```
Eg3. digitalRead(2);
```

```
digitalWrite(pin, value);
```

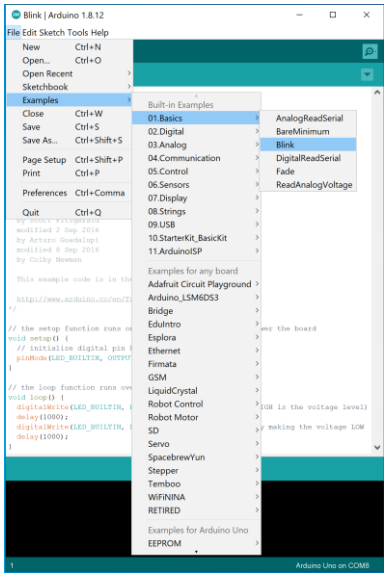
Writes `HIGH` or `LOW` to a pin

```
Eg2. digitalWrite(13, HIGH);
```

Our first Arduino Sketch/Program

```
/*
 *  Arduinos ketch to toggle the LED connected to pin-13 with a rate/delay of 1sec
 */
void setup()
{
  // put your setup code here, to run once:                    -->I*
  pinMode(13, OUTPUT);  //pin-13 configures as o/p            -->II
}
void loop()
{
  // put your main code here, to run repeatedly:                          -->1*
  digitalWrite(13, HIGH);      //HIGH Value or Bunary-1 send to pin-13    -->2
  //delay(x);                  //x-ms second(s) delay                     -->3*
  //delayMicroseconds(y);      //y-us second(s) delay                     -->4*
  delay(1000);                 //1000-milliseconds=1second delay     -->5
  digitalWrite(13, LOW);       //LOW Value or Bunary-1 send to pin-13     -->6
  delay(1000);                 //1000-milliseconds=1second delay     -->7
  //Toggling rate of led connected to pin-13 is of 1second               -->8*
}
```

Uploading and Running the blink sketch



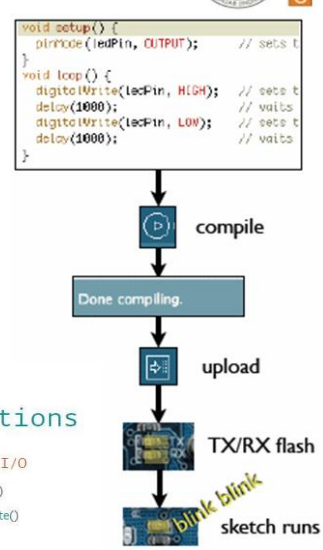
In Arduino, open up:

File → Examples → 01.Basics → Blink

```
const int ledPin = 13; // LED connected to
digital pin 13
void setup()
{
pinMode(ledPin, OUTPUT);
}
void loop()
{
digitalWrite(ledPin, HIGH); // set the LED on
delay(2000); // wait for two seconds
digitalWrite(ledPin, LOW); // set the LED off
delay(2000); // wait for two seconds
}
```

Uploading and Running the blink sketch (Cont..)

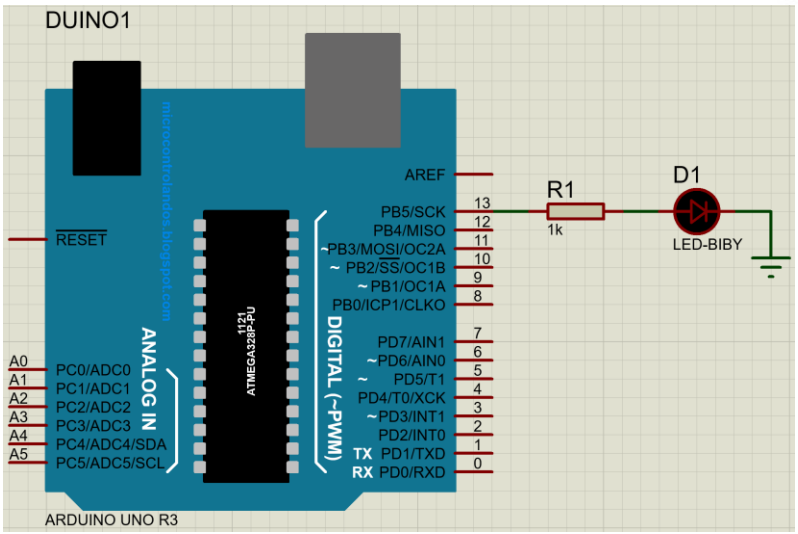
- Write your sketch/program
- Press compile button
  (to check for errors)
- Press upload button to program
  Arduino board with your sketch
- Try it out with the "Blink" sketch!



Arduino programs can be divided in three main parts: Values(Variables & Constants), Structure, and functions.

Structure	Variables	Functions
- setup()	Constants	Digital I/O
- loop()	- HIGH | LOW	- pinMode()
Control Structures	- INPUT | OUTPUT | INPUT_PULLUP	- digitalWrite()

Proteus design



Arduino Data Types

- Except in situations where maximum performance or memory efficiency is required, variables declared using int will be suitable for numeric values if the values do not exceed the range (shown in the first row in below Table) and if you don't need to work with fractional values.
- Most of the official Arduino example code declares numeric variables as int.
- But sometimes you do need to choose a type that specifically suits your application.

Numeric types	Bytes	Range	Use
`int`	2	-32768 to 32767	Represents positive and negative integer values.
`unsigned int`	2	0 to 65535	Represents only positive values; otherwise, similar to `int`.
`long`	4	-2147483648 to 2147483647	Represents a very large range of positive and negative values.
`unsigned long`	4	4294967295	Represents a very large range of positive values.

Arduino Data Types (Cont..)

- Sometimes you need negative numbers and sometimes you don't, so numeric types come in two varieties: signed and unsigned.
- unsigned values are always positive. Variables without the keyword unsigned in front are signed so that they can represent negative and positive values.
- One reason to use unsigned values is when the range of signed values will not fit the range of the variable (an unsigned variable has twice the capacity of a signed variable).
- Another reason programmers choose to use unsigned types is to clearly indicate to people reading the code that the value expected will never be a negative number.

Arduino Data Types (Cont..)

Numeric types	Bytes	Range	Use
`float`	4	3.4028235E+38 to -3.4028235E+38	Represents numbers with fractions; use to approximate real-world measurements.
`double`	4	Same as `float`	In Arduino, `double` is just another name for `float`.
`boolean`	1	`false` (0) or `true` (1)	Represents true and false values.
`char`	1	-128 to 127	Represents a single character. Can also represent a signed value between -128 and 127.
`byte`	1	0 to 255	Similar to `char`, but for unsigned values.
Other types			
`string`	Represents arrays of `chars` (characters) typically used to contain text.		
`void`	Used only in function declarations where no value is returned.		

Arduino Data Types (Cont..)

- Boolean types have two possible values: true or false. They are commonly used for things like checking the state of a switch (if it's pressed or not).
- You can also use HIGH and LOW as equivalents to true and false where this makes more sense;
  - digitalWrite(pin, HIGH) is a more expressive way to turn on an LED than digitalWrite(pin, true) or digitalWrite(pin,1), although all of these are treated identically when the sketch actually runs.

Data Types and Operators

Integer: used with integer variables with value between 2147483647 and -2147483647.

Eg: int x=1200;

Character: used with single character, represent value from - 127 to 128.

Eg. char c='r';

Long: Long variables are extended size variables for number storage, and store 32 bits (4 bytes), from -2,147,483,648 to 2,147,483,647.

Eg. long u=199203;

Floating-point numbers can be as large as 3.4028235E+38 and as low as -3.4028235E+38. They are stored as 32 bits (4 bytes) of information.

Eg. float num=1.291; [The same as **double** type]

Statements and operators:

Statement represents a command, it ends with **;**
Eg: **int x; x=13;**

Operators are symbols that used to indicate a specific function:

Math operators: [**+,-,*,/,%,^**]

Logic operators: [**==, !=, &&, ||**]

Comparison/Boolean operators: [==, >, <, !=, <=, >=]

Syntax:

; Semicolon, **{}** curly braces,

// single line comment,
/**Multi-line comments**/

Boolean Operators:
== (is equal?)
!= (is not equal?)
> (greater than)
>= (greater than or equal)
< (less than)
<= (less than or equal)

Compound Operators:
++ (increment)
-- (decrement)
+= (compound addition)
-= (compound subtraction)
*= (compound multiplication)
/= (compound division)

Control statements:

If Conditioning:

```
if(condition)
{
statements-1;
…
Statement-N;
}
else if(condition2)
{
Statements;
}
else
{
statements;
}
```

Switch case:

```
switch (var)
{
        case 1:
//do something when var equals 1  break;
case 2:
//do something when var equals 2  break;
default:
// if nothing else matches, do the default
// default is optional
}
```

Loop statements:

Do… while:

```
do
{
Statements;
}
while(condition);    // the statements are run at least once.
```

While:

```
While(condition)
{ statements; }
```

for

```
for (int i=0; i <= val; i++)
{
statements;
}
```

Use *break* statement to stop the loop whenever needed.

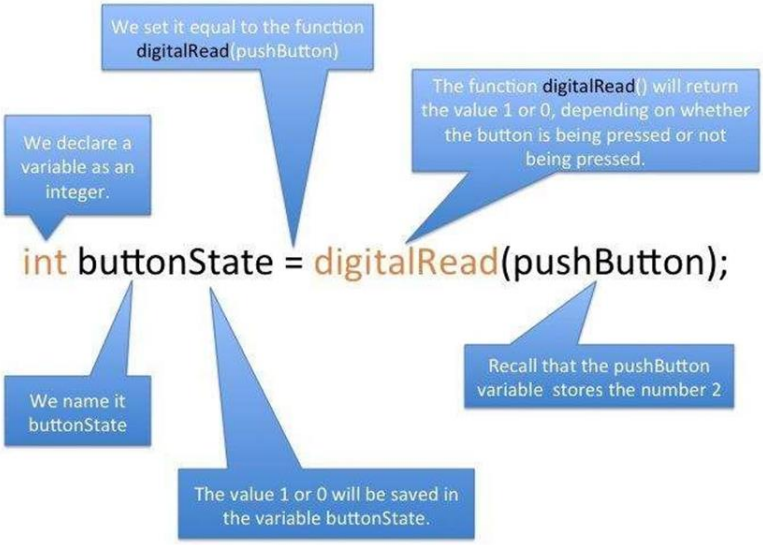
Digital Input

- Connect digital input to your Arduino using Pins # 0 − 13 (Although pins # 0 & 1 are also used for programming)
- Digital Input needs a `pinMode` command: `pinMode (pinNumber, INPUT);` *Make sure to use ALL CAPS for **INPUT***
- To get a digital reading:

```
int buttonState = digitalRead (pinNumber);
```

- Digital Input values are only **HIGH** (On) or **LOW** (Off)

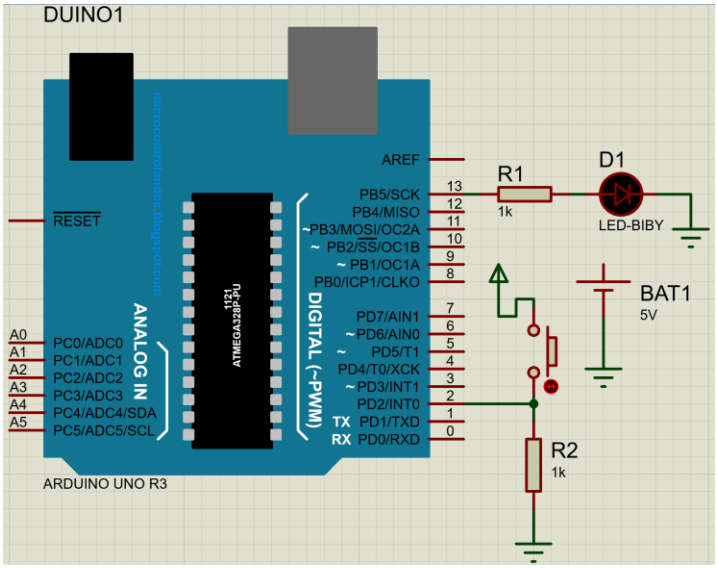
Digital Input (Cont..)



// Pushbutton sketch: a switch is connected to pin 2 lights the LED on pin 13
// (or) Control an LED connected to pin-13 w.r.to. a switch connected to pin-2, with the help of external resister

```
const int ledPin = 13; // choose the pin for the LED
const int inputPin = 2; // choose the input pin (for a pushbutton)
void setup()
{
pinMode(ledPin, OUTPUT); // declare LED as output
pinMode(inputPin, INPUT); // declare pushbutton as input
}
void loop()
{
int val = digitalRead(inputPin); // read input value
        if (val == HIGH) // check if the input is HIGH
        {
        digitalWrite(ledPin, HIGH); // turn LED on if switch is pressed
        }
        else
        {
        digitalWrite(ledPin, LOW); // turn LED off
        }
}
```

Proteus design (with external resister)



Using a key/push button with external resistor

- The digitalRead function monitors the voltage on the input pin (inputPin), and it returns a value of HIGH if the voltage is high (5 volts) and LOW if the voltage is low (0 volts).

- Actually, any voltage that is greater than 2.5 volts (half of the voltage powering the chip) is considered HIGH and less than this is treated as LOW.
- If the pin is left unconnected (known as *floating*) the value returned from digitalRead is indeterminate (it may be HIGH or LOW, and it cannot be reliably used).
- The resistor shown in Figure shown in previous slide ensures that the voltage on the pin will be low when the switch is not pressed, because the resistor “pulls down” the voltage to ground.
- When the switch is pushed, a connection is made between the pin and +5 volts, so the value on the pin interpreted by digital Read changes from LOW to HIGH.